

BS (SE) FINAL PROJECT

LINKGUARD (Fraud Link Detection Web- App)



PROJECT SUPERVISOR

Mr. Muhammad Majid

PRESENTED BY

Gul Ahmad (S22NSEEN1M01009)

**Faculty of Computing
Department of Software Engineering**

**The Islamia University of Bahawalpur
Bahawalnagar Campus**

Software Requirements Specification

Date (dd/mm/yyyy)	Version	Description
Current date	1.0	LinkGuard represents a comprehensive solution to the growing problem of phishing attacks. By combining machine learning algorithms with multiple threat intelligence sources, the system provides users with accurate and timely assessments of URL safety. The application's intuitive interface makes sophisticated security analysis accessible to users of all technical levels.

Team Members

Member Name	Primary Responsibility
Gul Ahmad	Full-stack development including frontend design, backend development, database design, system integration, testing, and documentation

Table of Contents

CHAPTER NO.1	1
INTRODUCTION	1
1.1 Overview	2
1.2 Project Significance	2
1.3 Scope and Objectives	2
1.3.1 Primary Objectives:	2
1.3.2 Target Users:	3
1.4 Core Functional Modules:	3
1.4.1 URL Analysis Module:	3
1.4.2 Machine Learning Module:	3
1.4.3 Threat Intelligence Module:	3
1.4.4 User Management Module:	3
1.4.5 Report Generation Module:	3
1.4.6 Image Processing Module:	3
CHAPTER NO.2 SYSTEM OVERVIEW	4
2.1 Introduction	5
2.2 URL Analysis and Detection	5
2.2.1 URL Analysis Engine	5
2.2.2 Real-time URL Analysis:	5
2.2.3 Machine Learning Prediction:	5
2.2.4 Threat Intelligence Integration:	5
2.3 User Authentication and Management	5
2.3.1 Secure Registration and Login:	5
2.3.2 Profile Management:	6
2.4 Advanced Features	6
2.4.1 Image-based URL Extraction:	6
2.4.2 Report Generation:	6
2.4.3 Security Analytics:	6
2.5 Agile Implementation	6
2.5.1 Sprint Planning:	6
2.5.2 Scrum Roles:	7
2.5.3 Continuous Integration:	7
CHAPTER NO. 3 REQUIREMENTS	8
3.1 Introduction	9
3.1.2 Requirements Importance:	9
3.2 Functional Requirements	9
3.2.1 User Authentication and Management	9
3.2.2 URL Analysis and Detection	9
3.2.3 Image Processing and URL Extraction	10
3.2.4 Threat Intelligence Integration	10
3.2.5 Report Generation and Management	10
3.2.6 Dashboard and Analytics	10
3.3 Non-Functional Requirements	10
3.3.1 Performance Requirements	11
3.3.2 Security Requirements	11
3.3.3 Usability Requirements	11
3.4 Use Cases	11
3.4.1 Use Case 1: User Registration	11
3.4.2 Use Case 2: URL Analysis	12
3.4.3 Use Case 3: Image Upload and URL Extraction	13
3.4.4 Use Case 4: Report Generation	14

3.4.5 Use Case 5: Dashboard Viewing	15
3.5 Use Case Diagram.....	16
3.6 Technical Requirements	17
3.6.2 Backend:	17
3.6.3 Database:	17
3.6.4 Machine Learning:	17
3.7 Security Requirements:	17
3.7.1 Authentication and Authorization:.....	17
3.7.2 Data Protection:	17
3.7.3 API Security:.....	17
CHAPTER NO.4.....	19
DESIGN AND ARCHITECTURE	19
4.1 Introduction	20
4.2 System Architecture Overview	20
4.2.1 Presentation Layer (Frontend).....	20
4.2.2 Business Logic Layer (Backend)	20
4.2.3 Data Layer.....	20
4.2.4 System Architecture Diagram.....	21
4.3 Modular Breakdown:	22
<i>Table 1: Modular Breakdown</i>	22
4.4 Process Flow / Representation.....	23
4.4.1 Major Process (Activity Diagram) Steps:.....	23
4.4.2 Process Flow:	24
4.4.3 Activity Diagram:	25
4.4.4 Login & Registration Activity Diagram:.....	26
.....	26
<i>Figure: Activity Diagram</i>	26
4.5 Frontend Architecture	27
4.5.1 Technology Stack:	27
4.5.2 Component Structure:	27
4.6 Backend Architecture.....	27
4.6.1 Technology Stack:	28
4.6.2 API Design:.....	28
4.6.3 Database Design:	28
4.7 Machine Learning Pipeline Architecture.....	28
4.7.1 Feature Extraction:	29
4.7.2 Model Implementation:	29
4.7.3 ML Pipeline Flowchart Explanation.....	29
4.8 Security Architecture	30
4.8.1 Authentication Flow:	30
4.8.2 Data Protection:	30
4.8.3 Authentication Flow Explanation	30
4.8.4 Integration Architecture.....	31
4.9 Design Models	31
4.9.1 Sequence Diagram:	31
4.9.2 Data Flow Diagram	33
4.10 Data Design	35
4.10.1 Storage Structure	35
4.10.2 Data Entities.....	36
4.10.3 Data Dictionary.....	37
CHAPTER NO.5 TESTING	39
5.1 Testing.....	40
5.1.1 Software Testing:	40
5.2 Testing Strategy	40
5.2.1 Unit Testing:	40

5.2.2 Integration Testing:.....	40
5.2.3 Performance Testing	41
5.2.4 Security Testing.....	41
5.2.5 User Acceptance Testing	41
5.2.6 System Testing:	42
5.2.7 Black Box Testing:	42
5.2.8 White Box Testing:	42
5.2 Test Cases.....	42
5.2.1 Frontend Testing:.....	42
5.2.2 Backend Testing:.....	42
5.2.3 Performance Testing:	43
5.3 Security Testing	43
5.4 Test Coverage Report Explanation	43
CHAPTER NO.6 GRAPHICAL USER INTERFACE	45
6.1 Introduction	46
6.2 Interface Screenshots and Descriptions.....	46
6.2.1 Home Page.....	46
6.2.2 Result Page:	48
.....	50
6.2.3 Registration Page.....	50
6.2.4 Login Page	51
6.2.5 User Dashboard.....	51
6.2.6 History Page.....	52
6.2.7 Mobile Interface.....	53
CHAPTER NO.7 CONCLUSION	55
7.1 Project Summary.....	56
7.1.1 Key Achievements:	56
7.2 Future Enhancement	56
CHAPTER NO. 8 REFERENCES	57
References:	58
Appendix	58

CHAPTER NO.1 INTRODUCTION

1.1 Overview

LinkGuard is an advanced **web-based security** application designed to combat phishing attacks through sophisticated URL analysis and threat intelligence integration. In an era where cybercrime costs the global economy over **\$6 trillion** annually, phishing remains one of the most prevalent attack vectors, accounting for over **80%** of reported security incidents. This project addresses this critical challenge by implementing a **multi-layered detection system** that combines machine learning algorithms with real-time threat intelligence feeds.

The system processes URLs through multiple detection layers, analyzes image content for embedded malicious links, and generates comprehensive security reports. Users receive instant feedback with confidence scores, enabling them to make informed decisions about potentially dangerous links before engaging with them.

1.2 Project Significance

Traditional security measures often rely on blacklists, which fail to detect new or sophisticated phishing campaigns. LinkGuard addresses this critical gap through its innovative approach:

Proactive Detection: Uses machine learning to identify suspicious patterns in URLs, even if they have never been seen before.

Real-Time Analysis: Provides instant feedback, crucial for preventing users from clicking on harmful links.

Multi-Vector Input: Analyzes URLs submitted via text and extracted from images, covering a wider range of attack vectors.

User Education: Goes beyond a simple "safe/unsafe" verdict by explaining the reasoning behind its classification, thus raising user awareness.

Comprehensive Reporting: Generates detailed security reports in Word formats, useful for individuals, businesses, and educational purposes.

1.3 Scope and Objectives

1.3.1 Primary Objectives:

- To develop a system capable of detecting phishing URLs with high accuracy achieved through an ensemble machine learning model.
- To provide real-time analysis results to users within few seconds.
- To implement an advanced image processing module capable of extracting and analyzing URLs from image files.
- To generate comprehensive security reports for users in Word formats.

- To ensure the system is secure, scalable, and user-friendly, with a responsive web interface.

1.3.2 Target Users:

- **Individual Users:** Anyone seeking to protect themselves from online scams and fraud.
- **Small & Medium Businesses:** Organizations without dedicated cybersecurity teams.
- **Educational Institutions:** For teaching cybersecurity awareness and best practices.
- **Cybersecurity Professionals:** As a quick and reliable tool for URL validation.

1.4 Core Functional Modules:

1.4.1 URL Analysis Module:

- Processes submitted URLs through multiple detection algorithms
- Integrates with external threat intelligence sources
- Generates comprehensive security assessments

1.4.2 Machine Learning Module:

- Implements stacking ensemble model for phishing detection
- Continuously learns from new phishing patterns
- Provides confidence scores for predictions

1.4.3 Threat Intelligence Module:

- Integrates with Google Safe Browsing API
- Connects to VirusTotal for malware detection
- Queries URLhaus for known malicious URLs
- Performs WHOIS analysis for domain information

1.4.4 User Management Module:

- Handles user registration, authentication, and profile management
- Implements secure session management with JWT tokens
- Provides role-based access control

1.4.5 Report Generation Module:

- Creates detailed security reports in PDF and Word formats
- Includes recommendations and security insights
- Customizes reports based on analysis results

1.4.6 Image Processing Module:

- Extracts URLs from uploaded images using OCR
- Enhances image quality for better text recognition
- Supports multiple image formats and processing techniques

CHAPTER NO.2 SYSTEM OVERVIEW

2.1 Introduction

The System Features section outlines the major functionalities and services that the LinkGuard application provides. Each feature is designed to enhance the security posture of users by providing comprehensive phishing detection capabilities.

2.2 URL Analysis and Detection

2.2.1 URL Analysis Engine

The core of LinkGuard is its powerful analysis engine, which processes URLs through a multi-stage detection pipeline:

- **Feature Extraction:** Analyzes the URL's structure, length, domain age, use of HTTPS, presence of suspicious characters, and more.
- **Machine Learning Prediction:** A trained stacking ensemble model (Random Forest, SVM, etc.) provides an initial classification.
- **Threat Intelligence Verification:** The system cross-references the URL against real-time databases from Google Safe Browsing, VirusTotal, and URLhaus.
- **WHOIS Analysis:** Gathers domain registration information to assess the credibility and age of the website.

Result Aggregation: Combines all findings into a final, weighted verdict with a confidence score.

2.2.2 Real-time URL Analysis:

- Instant assessment of submitted URLs
- Multi-layered detection approach combining ML and threat intelligence
- Confidence scoring to indicate detection certainty

2.2.3 Machine Learning Prediction:

- Stacking ensemble model for high accuracy
- Feature extraction from URL structure and content
- Continuous model improvement through feedback

2.2.4 Threat Intelligence Integration:

- Google Safe Browsing API for known malicious URLs
- VirusTotal integration for malware detection
- URLhaus database for phishing and malware URLs
- WHOIS analysis for domain reputation

2.3 User Authentication and Management

2.3.1 Secure Registration and Login:

- User account creation with email verification
- Secure password storage with hashing
- JWT-based authentication for session management

2.3.2 Profile Management:

- Personal dashboard with analysis history
- Customizable security preferences
- Account settings and password management
- Users can access and download their previously generated all links checked history from their dashboard.

2.4 Advanced Features

2.4.1 Image-based URL Extraction:

- OCR technology to extract URLs from images
- Multiple image enhancement techniques
- Support for various image formats

2.4.2 Report Generation:

- Detailed security reports in PDF and Word formats
- Customizable reports with recommendations
- Reports include the analyzed URL, the final verdict, confidence scores, a breakdown of threat intelligence findings, and personalized security recommendations
- Reports are tailored based on the analysis results (e.g., a "Phishing" report will have different recommendations than a "Safe" one)
- Download functionality for offline access
- Users can access and download their previously generated reports from their dashboard.

2.4.3 Security Analytics:

- Historical analysis of checked URLs
- Threat trend visualization
- Personal security statistics

2.5 Agile Implementation

The LinkGuard application follows an Agile development methodology with Scrum-based approach:

2.5.1 Sprint Planning:

- Sprint 1: User authentication and profile management
- Sprint 2: Core URL analysis functionality
- Sprint 3: Machine learning model integration
- Sprint 4: Threat intelligence API integration

Link Detection System

- Sprint 5: Image processing and OCR capabilities
- Sprint 6: Report generation and export features

2.5.2 Scrum Roles:

- Product Owner: Prioritizes features based on security needs
- Scrum Master: Facilitates development process and removes impediments
- Development Team: Implements features and ensures quality

2.5.3 Continuous Integration:

- Regular testing and integration of new features
- Automated testing for core functionalities
- Incremental feature rollouts with stakeholder feedback

CHAPTER NO. 3

REQUIREMENTS

3.1 Introduction

Requirements are detailed descriptions of what a system must do, the conditions under which it must operate, and the constraints it must satisfy. They serve as the foundation for system design, development, and testing. Requirements are typically categorized into:

1. **Functional Requirements:** Specify what the system should do (functions, features, behaviors)
2. **Non-Functional Requirements:** Define how the system should perform (performance, security, usability)
3. **Constraints:** Limitations or restrictions on the system (technical, business, regulatory)

3.1.2 Requirements Importance:

- Provide clear understanding of project scope
- Serve as basis for design and development
- Establish criteria for testing and validation
- Help manage stakeholder expectations
- Reduce project risks and misunderstandings

3.2 Functional Requirements

3.2.1 User Authentication and Management

- **FR-001:** The system shall allow users to create accounts with unique usernames and valid email addresses
- **FR-002:** The system shall require passwords to meet complexity requirements (minimum 8 characters, including uppercase, lowercase, numbers, and special characters)
- **FR-003:** The system shall verify user email addresses through a confirmation link sent to the registered email
- **FR-004:** The system shall provide secure login functionality with session management
- **FR-005:** The system shall implement password reset functionality through email verification
- **FR-006:** The system shall automatically log out users after 30 minutes of inactivity
- **FR-007:** The system shall allow users to update their profile information and preferences
- **FR-008:** The system shall provide role-based access control (User/Admin roles)

3.2.2 URL Analysis and Detection

- **FR-009:** The system shall accept URLs through text input fields
- **FR-010:** The system shall validate URL format before processing
- **FR-011:** The system shall analyze URLs using machine learning models
- **FR-012:** The system shall provide classification results (Safe, Phishing, Suspicious)
- **FR-013:** The system shall display confidence scores for each analysis (0-100%)
- **FR-014:** The system shall complete URL analysis within few seconds

Link Detection System

- **FR-015:** The system shall store analysis history for logged-in users
- **FR-016:** The system shall provide detailed explanations for classification results

3.2.3 Image Processing and URL Extraction

- **FR-017:** The system shall accept image uploads in JPG, PNG formats
- **FR-018:** The system shall limit image file size to 10MB
- **FR-019:** The system shall process images using OCR technology to extract text
- **FR-020:** The system shall identify and extract URLs from extracted text
- **FR-021:** The system shall analyze extracted URLs using the same detection engine
- **FR-022:** The system shall display extracted URLs with confidence scores
- **FR-023:** The system shall provide visual overlay showing detected URLs on images

3.2.4 Threat Intelligence Integration

- **FR-024:** The system shall integrate with Google Safe Browsing API for known threats
- **FR-025:** The system shall query VirusTotal for malware detection
- **FR-026:** The system shall check URLhaus database for known phishing URLs
- **FR-027:** The system shall perform WHOIS analysis for domain information
- **FR-028:** The system shall aggregate results from multiple threat intelligence sources
- **FR-029:** The system shall implement fallback mechanisms when external APIs are unavailable
- **FR-030:** The system shall display threat intelligence results in a unified format

3.2.5 Report Generation and Management

- **FR-031:** The system shall generate detailed security reports in PDF format
- **FR-032:** The system shall generate reports in Word document format
- **FR-033:** The system shall include analysis results, threat intelligence data, and recommendations in reports
- **FR-034:** The system shall allow users to customize report templates
- **FR-035:** The system shall provide report download functionality
- **FR-036:** The system shall maintain a history of generated reports
- **FR-037:** The system shall implement batch report generation for multiple URLs

3.2.6 Dashboard and Analytics

- **FR-038:** The system shall provide a personalized dashboard for logged-in users
- **FR-039:** The system shall display analysis history with filtering and search capabilities
- **FR-040:** The system shall show security statistics and trends
- **FR-041:** The system shall provide visual representations of security data
- **FR-042:** The system shall offer personalized security recommendations
- **FR-043:** The system shall allow users to export analysis data

3.3 Non-Functional Requirements

3.3.1 Performance Requirements

- **NFR-001:** The system shall respond to user interactions within few seconds
- **NFR-002:** The system shall support 100+ concurrent users
- **NFR-003:** The system is designed for high availability.
- **NFR-004:** The system shall complete URL analysis within few seconds
- **NFR-005:** The system shall load pages within few seconds on standard broadband connections

3.3.2 Security Requirements

- **NFR-006:** The system shall encrypt all sensitive data at rest and in transit
- **NFR-007:** The system shall comply with GDPR data protection regulations
- **NFR-008:** The system shall protect against OWASP Top 10 vulnerabilities
- **NFR-009:** The system shall implement rate limiting to prevent abuse
- **NFR-010:** The system shall conduct regular security audits
- **NFR-011:** The system shall log all security-relevant events

3.3.3 Usability Requirements

- **NFR-012:** The system shall provide an intuitive interface for non-technical users
- **NFR-013:** The system shall be responsive and work on mobile devices
- **NFR-014:** The system shall comply with WCAG 2.1 accessibility standards
- **NFR-015:** The system shall support multiple languages (English, Urdu, Spanish)
- **NFR-016:** The system shall provide clear error messages and guidance

3.4 Use Cases

Use cases describe how a system will be used from an end-user perspective. They capture the interaction between actors (users or external systems) and the system to achieve specific goals. Each use case includes:

- **Actors:** Who or what interacts with the system
- **Preconditions:** What must be true before the use case can start
- **Main Flow:** The step-by-step interaction between actors and the system
- **Alternative Flows:** Optional paths or exception handling
- **Postconditions:** What is true after the use case completes

3.4.1 Use Case 1: User Registration

- **Use Case ID:** UC-001
- **Use Case Name:** User Registration
- **Actor:** User
- **Description:** New user creates an account to access LinkGuard services
- **Preconditions:**

Link Detection System

- User has access to the internet
- User has a valid email address
- **Main Flow:**
 - User navigates to the registration page
 - User enters username, email, and password
 - System validates input format and requirements
 - System checks if username and email are unique
 - System creates user account with hashed password
 - System sends verification email to user
 - System displays confirmation message
 - User clicks verification link in email
 - System activates user account
 - User can now log in with credentials
- **Alternative Flows:**
 - If username or email already exists, system displays error message
 - If password doesn't meet requirements, system displays specific error
 - If verification link expires, user can request new verification email
- **Postconditions:**
 - User account is created and activated
 - User can log in to the system

3.4.2 Use Case 2: URL Analysis

- **Use Case ID:** UC-002
- **Use Case Name:** URL Analysis
- **Actor:** User
- **Description:** User submits a URL for phishing analysis
- **Preconditions:**
 - User is logged into the system
 - User has a valid URL to analyze
- **Main Flow:**
 - User navigates to the URL analysis page
 - User enters URL in the input field
 - System validates URL format
 - System extracts features from the URL
 - System processes URL through machine learning model

Link Detection System

- System queries threat intelligence APIs
- System aggregates results and calculates confidence score
- System displays analysis results with classification
- System provides detailed explanation of results
- System offers to generate a detailed report
- System saves analysis to user history
- **Alternative Flows:**
 - If URL format is invalid, system displays error message
 - If external APIs are unavailable, system uses cached data
 - If analysis takes longer than expected, system shows progress indicator
- **Postconditions:**
 - URL analysis is completed
 - Results are displayed to user
 - Analysis is saved to user history

3.4.3 Use Case 3: Image Upload and URL Extraction

- **Use Case ID:** UC-003
- **Use Case Name:** Image Upload and URL Extraction
- **Actor:** User
- **Description:** User uploads an image to extract and analyze URLs
- **Preconditions:**
 - User is logged into the system
 - User has an image file containing text
- **Main Flow:**
 - User navigates to the image analysis page
 - User uploads image file
 - System validates image format and size
 - System processes image with OCR technology
 - System extracts text from the image
 - System identifies potential URLs in the extracted text
 - System analyzes each extracted URL
 - System displays extracted URLs with analysis results
 - System provides visual overlay on image showing detected URLs
 - User can select specific URLs for detailed analysis
 - System saves analysis to user history

- **Alternative Flows:**
 - If image format is not supported, system displays error message
 - If image size exceeds limit, system displays error message
 - If OCR fails to extract text, system displays error message
 - If no URLs are found, system displays appropriate message
- **Postconditions:**
 - Image is processed and URLs are extracted
 - Extracted URLs are analyzed
 - Results are displayed to user
 - Analysis is saved to user history

3.4.4 Use Case 4: Report Generation

- **Use Case ID:** UC-004
- **Use Case Name:** Report Generation
- **Actor:** User
- **Description:** User generates a detailed security report for URL analysis
- **Preconditions:**
 - User is logged into the system
 - User has completed at least one URL analysis
- **Main Flow:**
 - User navigates to the reports section
 - User selects a previous analysis or generates a new one
 - User chooses report format (PDF or Word)
 - User selects report template (detailed, summary, custom)
 - System compiles analysis data and threat intelligence results
 - System generates report with recommendations
 - System provides download link for the report
 - User downloads the report
 - System saves report to user history
- **Alternative Flows:**
 - If user selects custom template, system displays customization options
 - If report generation fails, system displays error message
 - If report generation takes time, system shows progress indicator
- **Postconditions:**
 - Security report is generated

- User can download the report
- Report is saved to user history

3.4.5 Use Case 5: Dashboard Viewing

- **Use Case ID:** UC-005
- **Use Case Name:** Dashboard Viewing
- **Actor:** User
- **Description:** User views their personalized security dashboard
- **Preconditions:**
 - User is logged into the system
- **Main Flow:**
 - User navigates to the dashboard page
 - System displays recent analysis history
 - System shows security statistics and trends
 - System provides visual representations of data
 - System offers personalized security recommendations
 - User can filter and search analysis history
 - User can export data or generate reports
 - User can access account settings
- **Alternative Flows:**
 - If user has no analysis history, system displays welcome message
 - If user applies filters, system updates displayed data
- **Postconditions:**
 - User views their security dashboard
 - User can interact with their data

3.5 Use Case Diagram

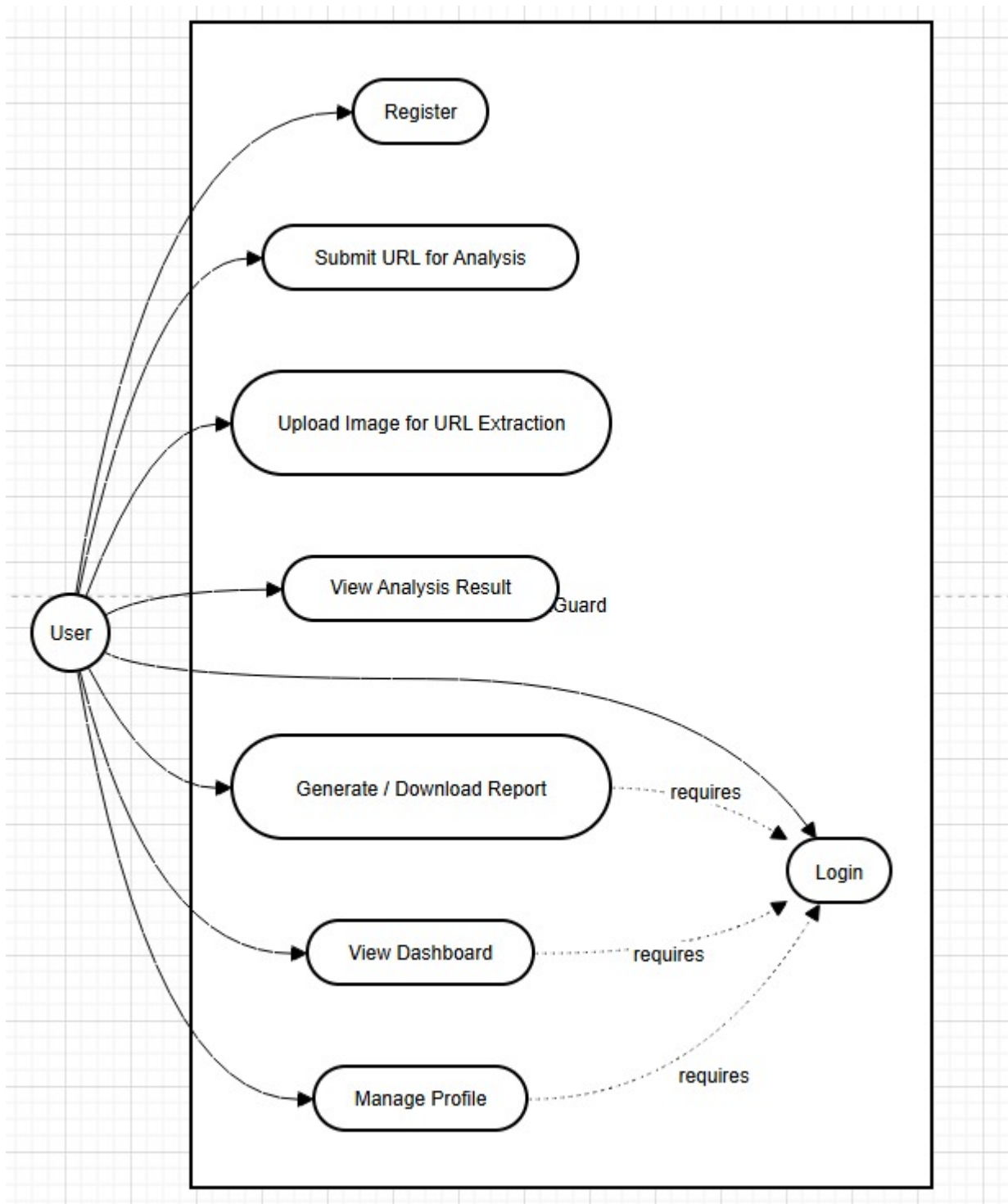


Figure 1: Use Case Diagram

3.6 Technical Requirements

3.6.1 Frontend:

- React.js framework for user interface
- Responsive design with Tailwind CSS
- State management with React Context
- Framer Motion for animations

3.6.2 Backend:

- Python Flask framework for API development
- SQLAlchemy for database management
- JWT for authentication
- Integration with external APIs

3.6.3 Database:

- SQLite for development environment
- PostgreSQL for production deployment
- Secure storage of user data and analysis history

3.6.4 Machine Learning:

- Scikit-learn for model implementation
- Joblib for model serialization
- Feature extraction from URL structures
- Stacking ensemble model for high accuracy

3.7 Security Requirements:

3.7.1 Authentication and Authorization:

- Secure login with multi-factor authentication
- Role-based access control
- Session management with token expiration
- Password strength requirements

3.7.2 Data Protection:

- Encryption of sensitive data at rest
- HTTPS for all communications
- GDPR compliance for user data
- Regular security audits

3.7.3 API Security:

- Rate limiting to prevent abuse

Link Detection System

- Input validation and sanitization
- Protection against injection attacks
- CORS configuration for secure frontend access

CHAPTER NO.4 DESIGN AND ARCHITECTURE

4.1 Introduction

System architecture is the conceptual model that defines the structure, behavior, and views of a system. It describes the components, their relationships, and the principles governing their design and evolution.

A well-defined architecture:

- Provides a blueprint for system development
- Ensures consistency across different parts of the system
- Facilitates communication among stakeholders
- Guides technology decisions and trade-offs
- Supports system maintenance and evolution
- Architectural Patterns Architectural patterns are reusable solutions to commonly occurring problems in software architecture. For LinkGuard, we've implemented:
- **Three-Tier Architecture:** Separation of presentation, business logic, and data layers
- **Microservices Pattern:** Modular design with independent, deployable services
- **Model-View-Controller (MVC):** Separation of concerns in frontend and backend
- **Repository Pattern:** Abstraction layer for data access
- **Observer Pattern:** For real-time updates and notifications

4.2 System Architecture Overview

Overall Architecture LinkGuard follows a three-tier architecture pattern with microservices approach:

4.2.1 Presentation Layer (Frontend)

- React.js application with component-based architecture
- Responsive UI components with Tailwind CSS
- State management with Context API and Redux
- API communication layer with Axios

4.2.2 Business Logic Layer (Backend)

- Flask REST API with modular design
- Authentication middleware with JWT
- Business logic implementation
- External API integrations with circuit breakers

4.2.3 Data Layer

- PostgreSQL database with connection pooling
- Redis for caching and session storage
- File storage with AWS S3

- Backup and disaster recovery systems

4.2.4 System Architecture Diagram

A System Architecture Diagram is a visual representation of a system's structure, showing its components, their relationships, and the interactions between them. It illustrates how the system is organized at a high level, including the major building blocks, their interfaces, and the principles that guide their design and evolution.

- **Used in This Project:**

For the LinkGuard project, the System Architecture Diagram is crucial for:

- Providing a clear visual overview of how the system is structured
 - Showing the separation of concerns between frontend, backend, and data layers
 - Illustrating the flow of data through the system
 - Depicting the integration with external services and APIs
 - Helping developers understand how different components interact
 - Assisting in identifying potential bottlenecks or points of failure
- **The diagram should show:**
 - Frontend (React.js) with its components
 - Backend (Flask) with its modules (Authentication, URL Analysis, Image Processing, Report Generation)
 - Database (SQLite/PostgreSQL)
 - External APIs (Google Safe Browsing, VirusTotal, URLhaus, WHOIS)
 - Data flow between components
 - Security layers (JWT authentication)

• Diagram:

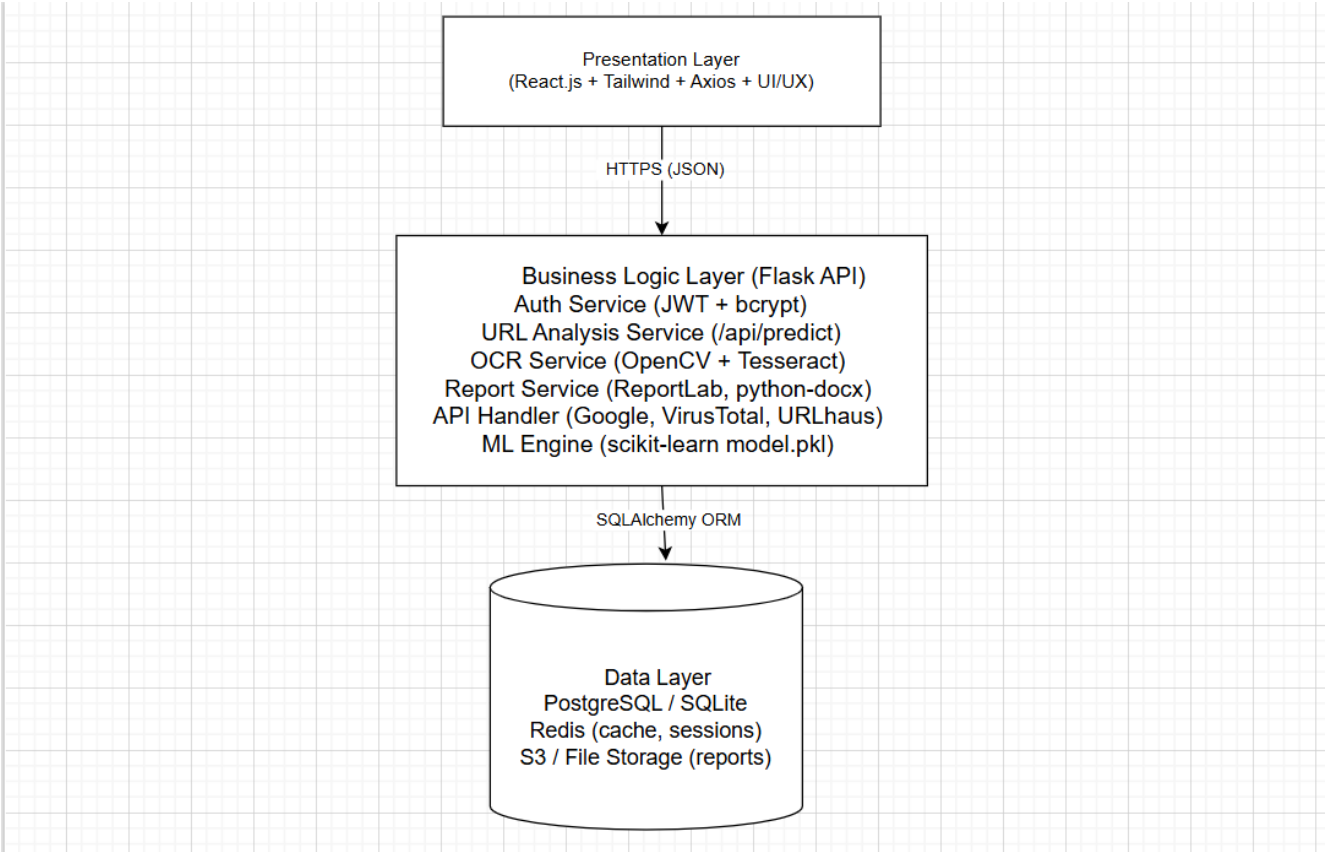


Figure 2: System Architecture

4.3 Modular Breakdown:

Table 1: Modular Breakdown

Module	Responsibility
User Interface	Accepts URL input, shows results (Safe, Suspicious, Malicious)
URL Feature Extractor	Extracts metadata (length, domain, HTTPS, etc.) from input
AI Model Engine	Classifies the URL using trained ML models
API Verifier	Queries WHOIS and TotalVirus for real-time threat verification

Result Generator	Combines AI + API response to create final result
-------------------------	---

4.4 Process Flow / Representation

The major process flow of the system is represented using an Activity Diagram, which outlines how a user's URL is processed, analyzed, verified, and responded to.

4.4.1 Major Process (Activity Diagram) Steps:

- User enters a URL in the mobile app
- URL is sent to backend
- System extracts features from URL
- AI model predicts result (Safe, Suspicious, Malicious)
- External APIs verify the URL (WHOIS, TotalVirus)
- Final decision is generated (based on AI + API)
- Result is displayed to the user

4.4.2 Process Flow:

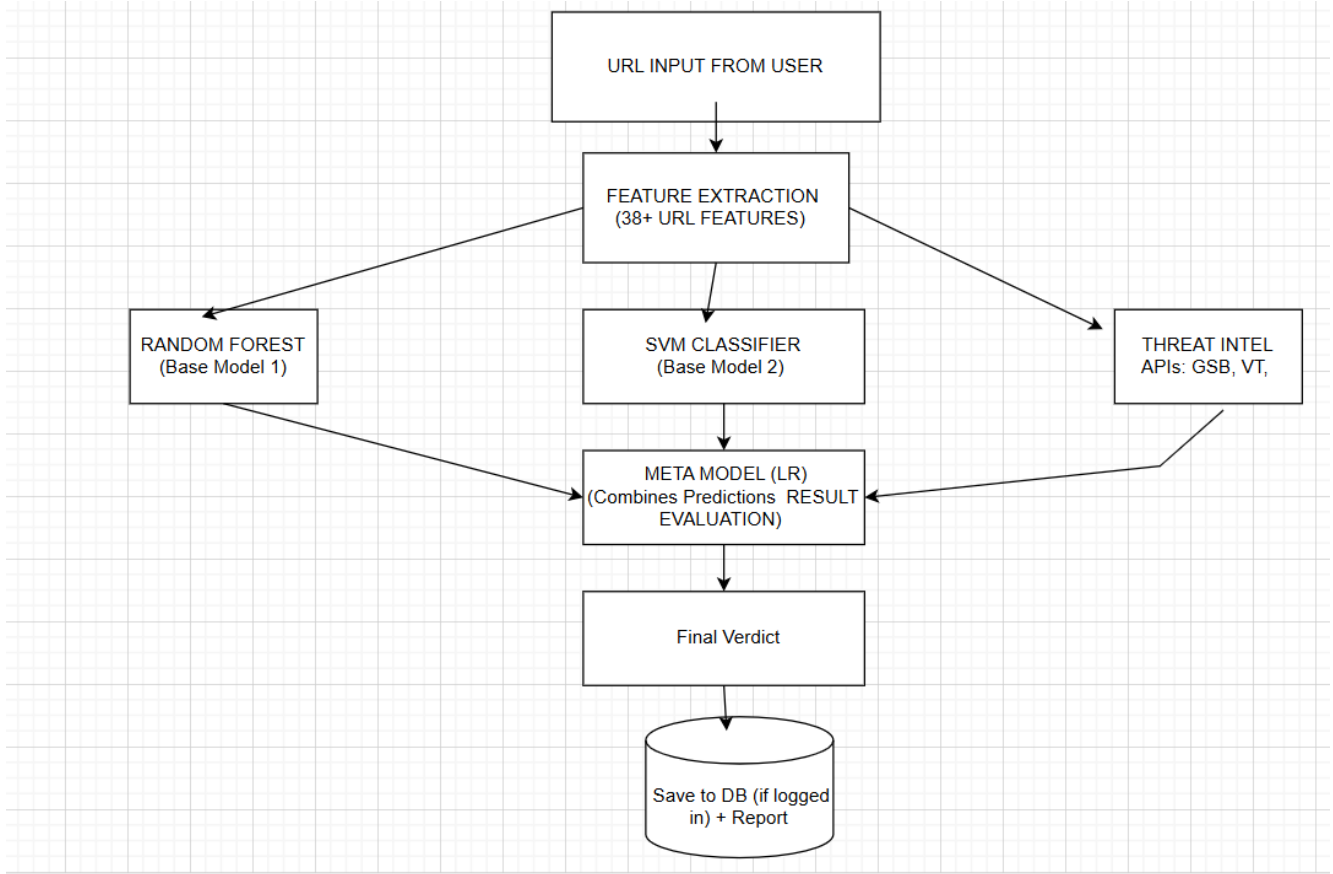


Figure 3: Process flow Diagram

4.4.3 Activity Diagram:

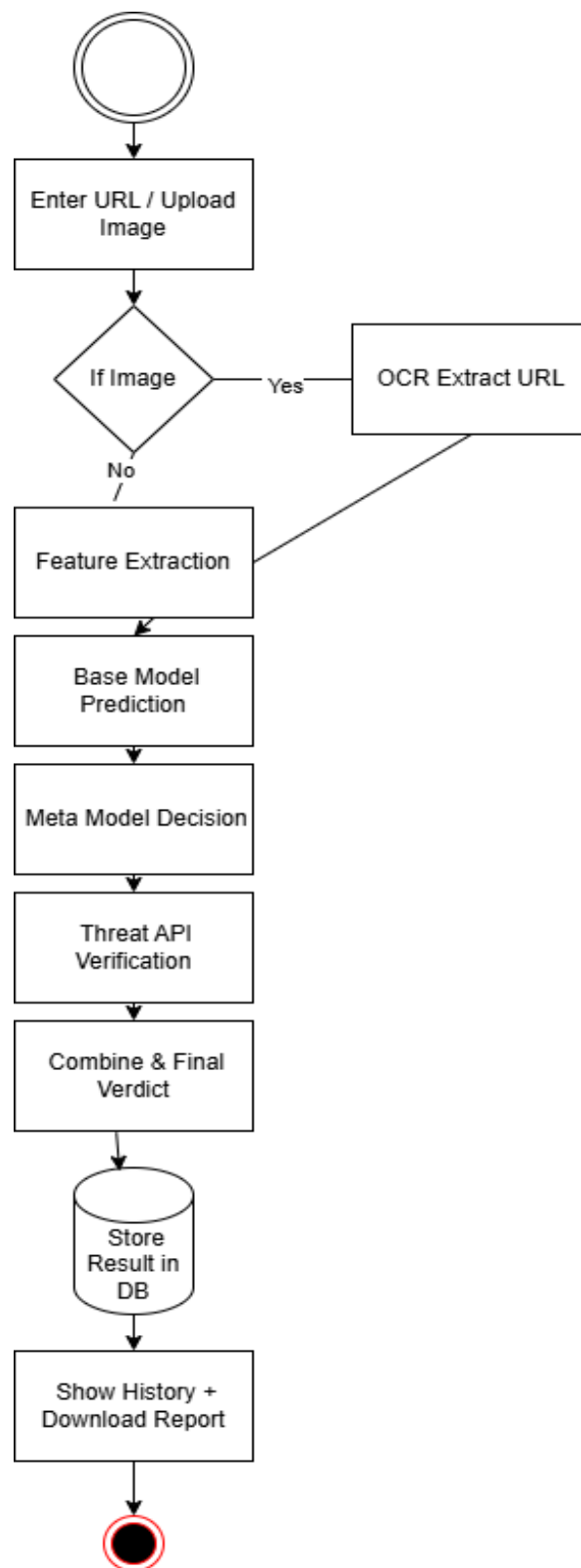


Figure 4: System Activity Diagram

4.4.4 Login & Registration Activity Diagram:

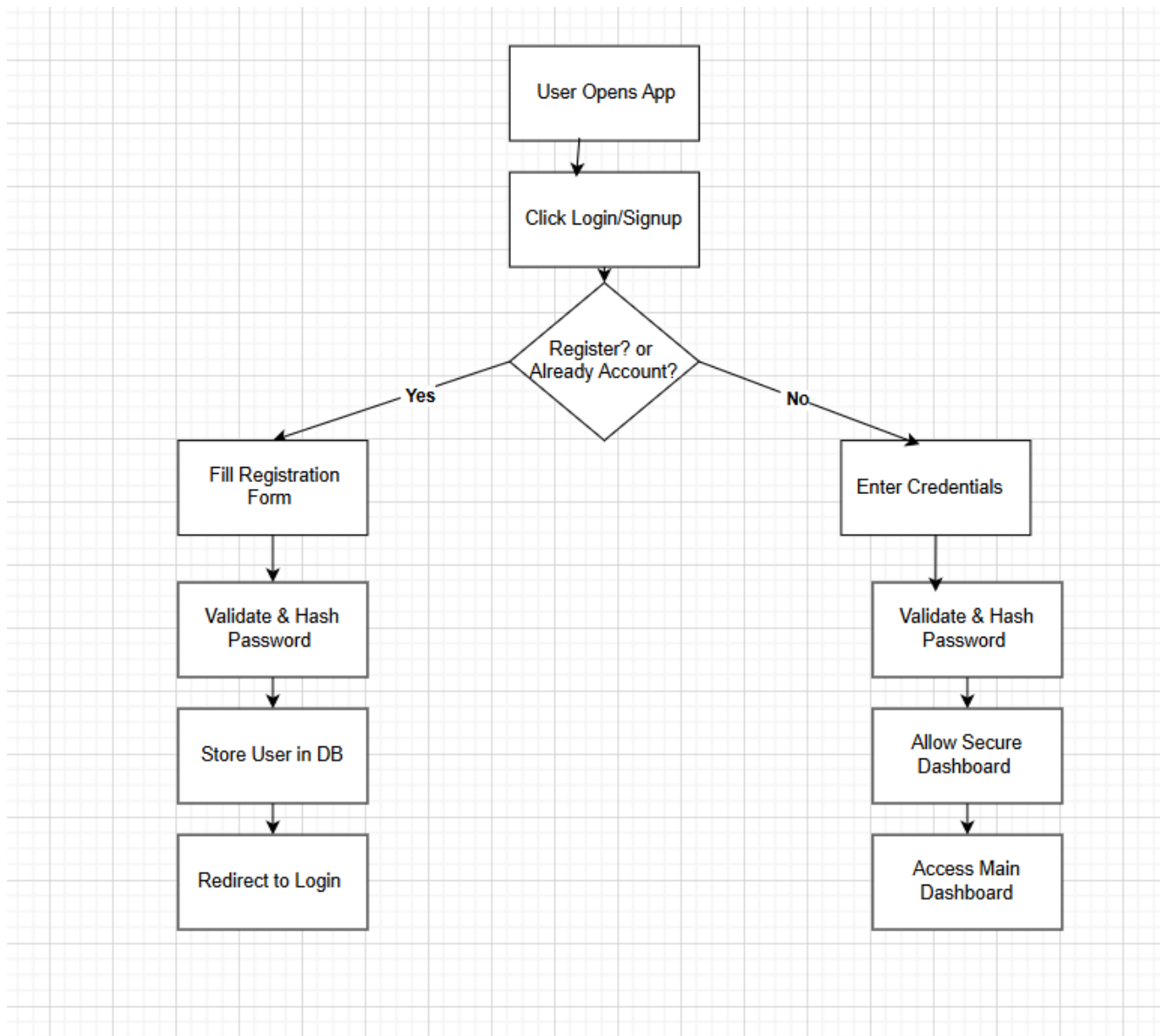


Figure: Activity Diagram

4.5 Frontend Architecture

The frontend is designed as a **Single-Page Application (SPA)** using a component-based architecture.

- **Routing:** Managed by **React Router**, with public routes (Home, Login, Register) and protected routes (Dashboard, History) that require authentication.
- **State Management:** Global state (like user authentication status) is managed using React's **Context API**. Local component state is managed with **useState** and **useReducer**.
- **API Communication:** All communication with the backend is handled through a centralized API service using **Axios**, which handles request/response interceptors for things like attaching JWT tokens.
- **UI Components:** The UI is broken down into reusable components (e.g., **NavBar**, **URLInput**, **ImageUpload**, **Spinner**) for consistency and maintainability.

4.5.1 Technology Stack:

- React.js for component-based UI development
- Tailwind CSS for responsive styling
- Framer Motion for animations
- Axios for API communication
- React Router for navigation

4.5.2 Component Structure:

- Reusable components for UI elements
- Custom hooks for state management
- Context API for global state
- Protected routes for authenticated pages

4.6 Backend Architecture

The backend follows a modular, service-oriented design within the Flask framework.

- **API Endpoints:** The application exposes a clean set of RESTful endpoints (e.g., `/api/auth/register`, `/api/predict`, `/api/extract-url-from-image`, `/api/download-report/<format>`).
- **Authentication:** A middleware decorator (**@token_required**) protects sensitive endpoints by validating JWT tokens from the **Authorization** header.
- **Core Services:** The logic is separated into distinct services:
 - **URLAnalyzer:** Handles feature extraction and orchestrates the prediction process.
 - **AIModel:** Wraps the trained scikit-learn model for making predictions.
 - **APIHandler:** Manages communication with external threat intelligence APIs.

- **ReportGenerator:** Contains the logic for creating PDF and Word documents.
- **Database Models:** SQLAlchemy models (**User**, **UrlCheck**) define the database schema and handle all database interactions in an object-oriented way.

4.6.1 Technology Stack:

- Python Flask for REST API development
- SQLAlchemy for database ORM
- JWT for authentication
- Integration with external APIs
- Machine learning model integration

4.6.2 API Design:

- RESTful endpoints for all functionalities
- JSON format for request/response
- Error handling with appropriate status codes
- Authentication middleware for protected routes

4.6.3 Database Design:

- **User Table:**
 - User credentials and profile information
 - Preferences and settings
 - Relationship with URL checks
- **URL Check Table:**
 - Analysis results and timestamps
 - Feature extraction data
 - User association for history tracking

4.7 Machine Learning Pipeline Architecture

The ML pipeline is designed for accuracy and performance.

- **Feature Extraction:** When a URL is submitted, a set of 15+ features are extracted, such as URL length, domain age, presence of 'https', number of subdomains, and use of suspicious keywords.
- **Feature Scaling:** The extracted features are scaled using a pre-fitted **StandardScaler** to ensure they are on a comparable scale for the model.
- **Prediction:** The scaled features are fed into a pre-trained **Stacking Ensemble Model**. This model combines the predictions of several base classifiers (like Random Forest and SVM) using a meta-classifier (Logistic Regression) to produce a final, highly accurate prediction.

- **Confidence Scoring:** The models **predict_proba** method is used to generate a confidence score, which is displayed to the user.

4.7.1 Feature Extraction:

- URL structure analysis
- Domain reputation metrics
- Content-based features
- Historical data patterns

4.7.2 Model Implementation:

- Stacking ensemble approach
- Multiple base classifiers
- Meta-classifier for final prediction
- Confidence score calculation
- Base Model

- **Random Forest** Mathematical Equation:

$$F_RF(x) = \text{mode}(\{T_1(x), T_2(x), \dots, T_n(x)\})$$

- **Support Vector Machine** Mathematical Equation:

$$f_SVM(x) = \text{sign}(w^T x + b)$$

- Meta Model

- **Logistic Regression** Mathematical Equation:

$$P(y=1|z) = \sigma(w^T z + b) = 1 / (1 + e^{-(w_1 p_RF + w_2 p_SVM + b)})$$

4.7.3 ML Pipeline Flowchart Explanation

An ML Pipeline Flowchart is a visual representation of the sequential steps involved in processing data through a machine learning model. It illustrates how raw data is transformed into predictions, showing each stage of the pipeline from input to output.

- **Used in This Project**

For the LinkGuard project, the ML Pipeline Flowchart is important because:

- It clarifies the complex process of URL analysis into understandable steps
- It shows how different components of the ML system work together
- It helps identify potential optimization points in the pipeline
- It provides a reference for implementation and debugging
- It makes the ML process accessible to non-technical stakeholders

- **ML Pipeline Flowchart for LinkGuard:**

URL Input -> Feature Extraction -> Feature Scaling -> ML Model -> Prediction/Confidence Score.

The flowchart should show:

- **Input:** URL
- **Process 1:** Feature Extraction (with examples of features)
- **Process 2:** Feature Scaling
- **Process 3:** ML Model (Stacking Ensemble)
- **Output:** Prediction (Safe/Phishing) with Confidence Score
- **Optional:** Threat Intelligence Check as a parallel process

4.8 Security Architecture

Security is a primary concern in the design.

- **Authentication:** Secure password hashing using **werkzeug.security**. JWTs are used for stateless authentication, with short-lived access tokens and longer-lived refresh tokens.
- **Authorization:** Role-based access control is implemented, ensuring users can only access their own data.
- **Data Protection:** All API communication is done over HTTPS. Sensitive data in the database (like passwords) is hashed. The system is designed to be GDPR-compliant.
- **Input Validation:** All user inputs, both on the frontend and backend, are rigorously validated and sanitized to prevent attacks like XSS and SQL Injection.

4.8.1 Authentication Flow:

- JWT-based authentication
- Token refresh mechanism
- Session management
- Password hashing with bcrypt

4.8.2 Data Protection:

- Encryption of sensitive data
- Secure API communication
- Input validation and sanitization
- Protection against common vulnerabilities

4.8.3 Authentication Flow Explanation

An Authentication Flow Diagram is a visual representation of the process by which a user's identity is verified and access is granted to a system. It shows the sequence of steps from initial login attempt to authenticated session, including the exchange of tokens and credentials.

- **Used in This Project:**

For the LinkGuard project, the Authentication Flow Diagram is essential because:

- It illustrates the security mechanisms implemented in the system
- It shows how JWT tokens are used for stateless authentication

Link Detection System

- It clarifies the login and registration processes
- It helps identify potential security vulnerabilities
- It serves as documentation for implementing and maintaining the authentication system

- **Authentication Flow Diagram for LinkGuard:**

JWT process: User sends credentials -> Backend validates -> Backend returns JWT -> Frontend stores JWT -> Frontend sends JWT with subsequent requests -> Backend validates JWT.

The Flow should show:

- **Login flow:** User enters credentials -> API request -> Validation -> JWT generation -> Response
- **Token storage:** Where JWT is stored on the client
- **Protected resource access:** Request with JWT -> Validation -> Grant/Deny access
- Token refresh process
- Logout flow

4.8.4 Integration Architecture

- **External APIs:**
 - Google Safe Browsing integration
 - VirusTotal API connection
 - URLhaus database access
 - WHOIS information retrieval
- **Image Processing:**
 - OCR technology for URL extraction
 - Image enhancement techniques
 - Multiple processing methods
 - Error handling for invalid inputs

4.9 Design Models

This section presents the visual design models used in the development of the **AI-Driven Fraudulent Link Detection System**, which follows the **Object-Oriented Design (OOD)** methodology. These diagrams help in visualizing the system architecture, interactions between objects, data movement, and behavior across different states.

4.9.1 Sequence Diagram:

The Sequence Diagram is a dynamic behavioral diagram in UML that shows how objects interact with each other in a particular sequence of events. It maps the flow of logic for a specific use case by representing objects, the messages exchanged, and the order of message transmission. In our system, the

Link Detection System

sequence diagram outlines the process of a user submitting a URL, the system validating it, and the result being returned — capturing the time-ordered interactions across components. The sequence diagram shows how objects interact **in a specific order** to fulfill a user request, such as URL verification.

- **Diagram:**

Shows how objects interact when a user submits a URL.

Flow: User → Frontend → Backend → AIModel → External API → Result

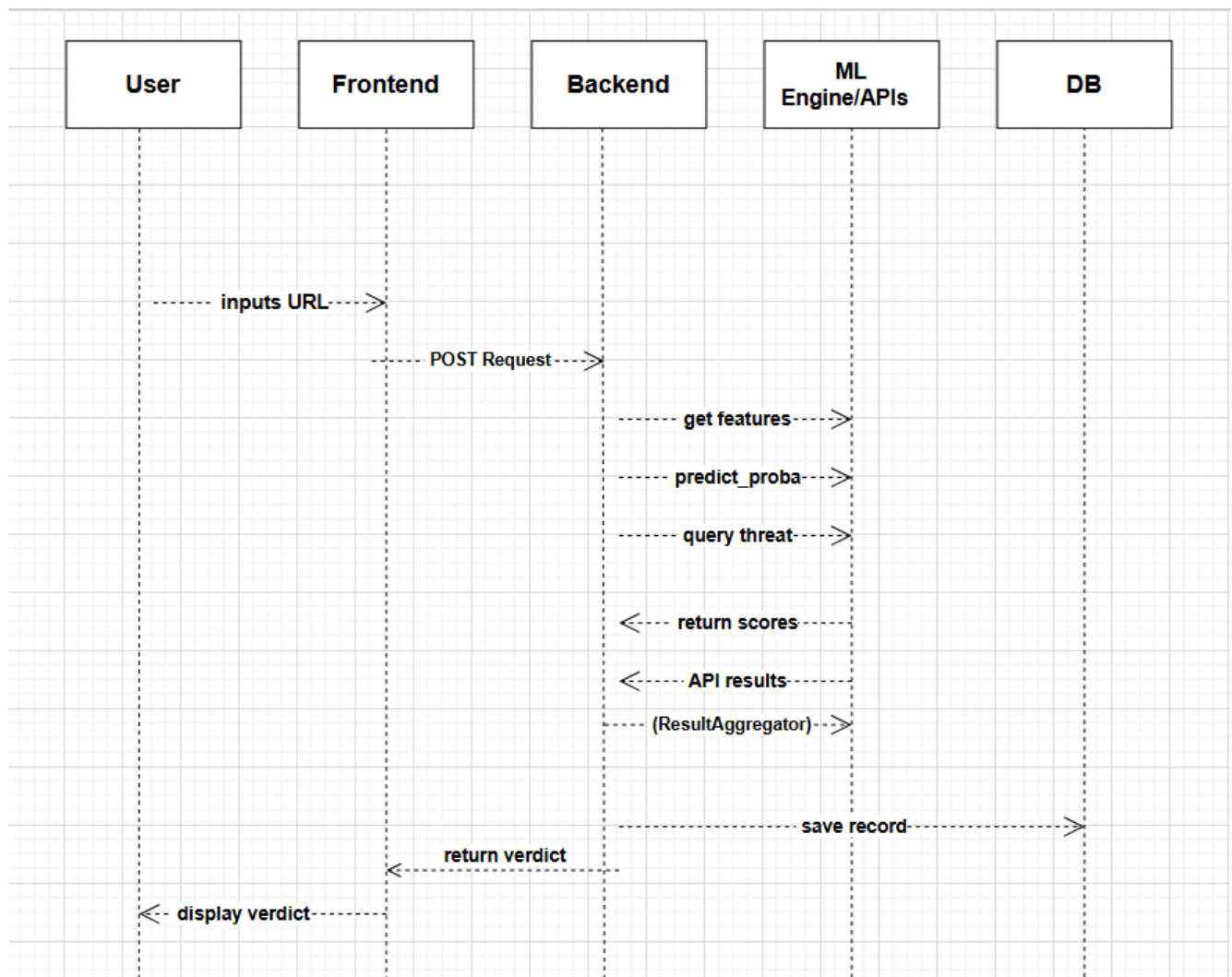


Figure 5: Sequence Diagram

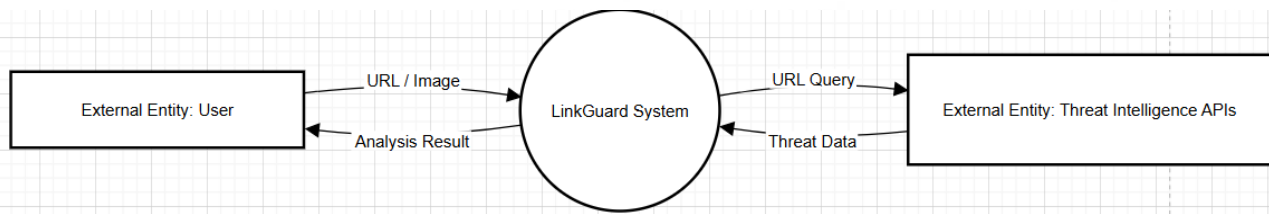
4.9.2 Data Flow Diagram

Since data movement is a critical part of this system (especially with AI and APIs), **DFDs are included to show how data flows** through different levels of the application.

- **Level 0:** Shows high-level interaction between **User**, **System**, and **External APIs**.



Dataflow Diagram (Level 0)

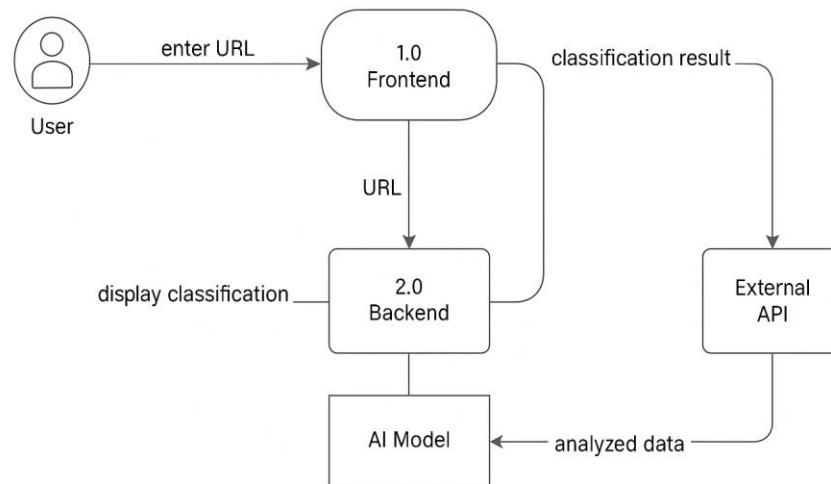
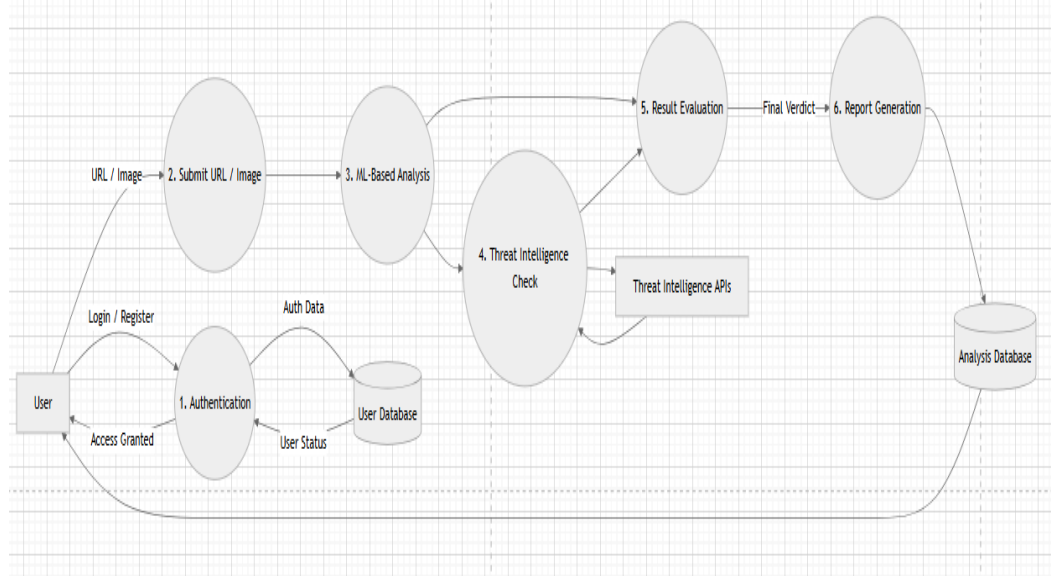


- **Level 1:** In Level 1, we decomposed the main system process into 3 primary subprocesses:

Processes:

- 1.2 URL Validation – Accepts the input URL and checks for basic structure and formatting.
- 1.3 Display Result – Takes classification output and displays it to the user.
- 1.4 External Verification (API) – Interacts with WHOIS and TotalVirus to check URL legitimacy.

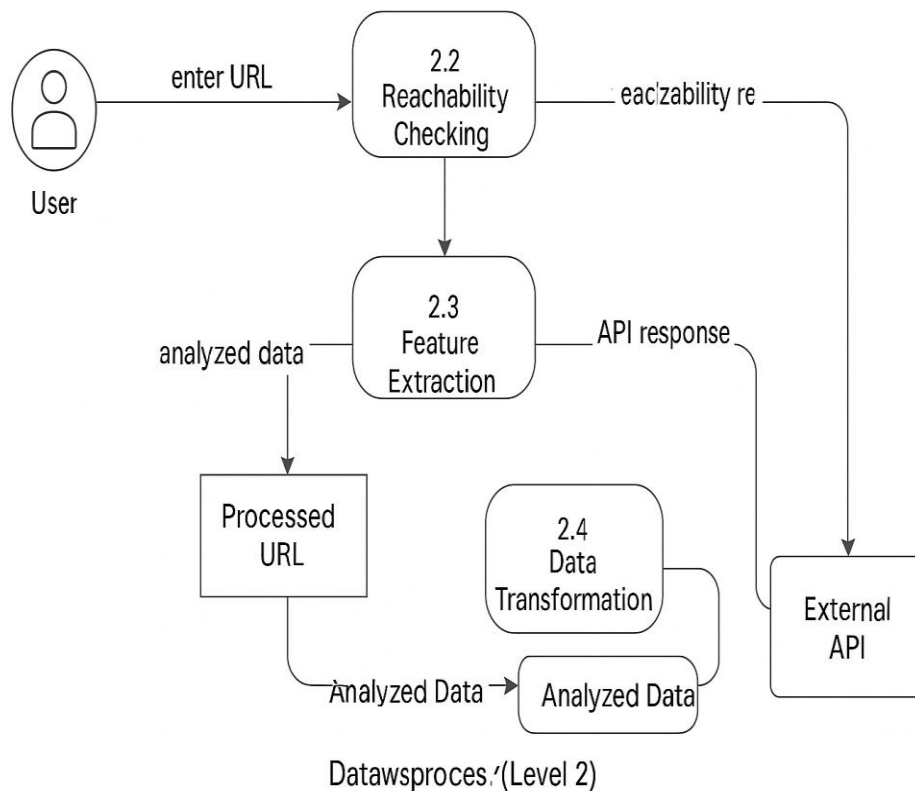
Link Detection System



- **Level 2:**

Expands on:

- How the URL is analyzed
- How features are extracted
- How responses are merged (AI + API)



4.10 Data Design

The **data design** of this system outlines how data is captured, processed, stored, and retrieved across different modules of the application. Since the system relies heavily on **AI-based classification** and **real-time URL verification**, data flows between several components: frontend (user), backend (Flask API & AI model), external APIs (WHOIS & TotalVirus), and a cloud database (**SQLite** or **MySQL**). All critical data such as URLs, classification results, feature sets, user history, and logs are stored in the database. The design ensures **data consistency, accuracy, and security**, particularly when dealing with sensitive or potentially harmful URLs.

4.10.1 Storage Structure

- **Frontend (React + Vite):** Captures input (URLs), displays results, sends/receives data from the backend.
- **Backend (Flask/Django):** Processes URL, performs AI prediction and API verification.
- **Cloud Database (SQLite / MySQL):**
 - 1) Stores URL records with results
 - 2) Maintains user verification history

- 3) Logs admin activities like retraining or blacklist updates

4.10.2 Data Entities

Table 2: Data Entities

Entity	Description
user_id	Unique identifier for each user (auto-generated or Firebase auth UID)
url	The website link submitted for verification
features[]	Array of metadata extracted from the URL (e.g., domain age, HTTPS, etc.)
result	Final classification (Safe, Suspicious, Malicious)
timestamp	Date and time when the URL was submitted
api_response	Combined result from WHOIS and TotalVirus APIs
history[]	List of previously submitted URLs with results (linked to user_id)
blacklist[]	Admin-maintained list of known malicious URLs
whitelist[]	Admin-maintained list of trusted URLs
logs[]	Activity logs (admin retraining, user submissions, etc.)

4.10.3 Data Dictionary

Table 3: Data Dictionary

Name	Type	Description
user_id	String	Unique ID of user
url	String	Website URL submitted by user
result	String	Safe / Phishing / Suspicious
features[]	Array	Features extracted from URL
timestamp	DateTime	Date and time of URL submission

CHAPTER NO.5

TESTING

5.1 Testing

Testing is an essential part of the software development lifecycle. In this chapter, we will outline the testing approach, types of tests, and the tools to be used for the **LinkGuard web-based app** project to ensure that it works as intended and meets the required standards for quality, performance, and security. Testing in software development involves evaluating a system to ensure it works as expected. It includes activities like checking individual components, ensuring they integrate well, testing the entire system, verifying acceptance criteria, checking for performance and security issues, and assessing usability. Testing can be manual or automated and aims to find defects early to improve software quality and reduce costs.

5.1.1 Software Testing:

Software testing is the process of evaluating and verifying that a software application meets its specified requirements and is free of defects. It is a critical phase in the software development lifecycle that ensures the quality, reliability, and performance of the final product.

5.2 Testing Strategy

Testing for the **LinkGuard** aims to ensure that the platform is reliable, secure, user- friendly, and performs efficiently under various conditions. Both manual and automated testing approaches will be used to validate the system's functionality, usability, performance, and security.

A multi-level testing strategy is employed to ensure the system's robustness:

5.2.1 Unit Testing:

Individual functions and components are tested in isolation. For example, testing the **extract_features** function or a React component's rendering.

- **Backend Testing:**
 - Test individual API endpoints
 - Validate business logic
 - Test database operations
 - Verify authentication mechanisms
- **Frontend Testing:**
 - Component rendering tests
 - User interaction testing
 - State management validation
 - Navigation flow testing

5.2.2 Integration Testing:

Testing how different parts of the system work together. For example, testing the interaction between the frontend and the **/api/predict** endpoint.

- **API Integration:**
 - Test frontend-backend communication
 - Validate data flow between components
 - Test error handling mechanisms
 - Verify authentication flows
- **External API Testing:**
 - Mock external API responses
 - Test integration with threat intelligence sources
 - Validate error handling for API failures
 - Test rate limiting and timeout handling

5.2.3 Performance Testing

- **Load Testing:**
 - Simulate multiple concurrent users
 - Test system under heavy load
 - Identify performance bottlenecks
 - Optimize response times
- **Stress Testing:**
 - Test system limits
 - Verify graceful degradation
 - Test recovery from failures
 - Validate resource management

5.2.4 Security Testing

- **Vulnerability Assessment:**
 - Penetration testing for security flaws
 - OWASP Top 10 vulnerability checks
 - Authentication bypass testing
 - Input validation testing
- **Data Protection Testing:**
 - Verify encryption implementation
 - Test secure communication protocols
 - Validate access control mechanisms
 - Test data sanitization processes

5.2.5 User Acceptance Testing

Testing the system from the end-user's perspective to ensure it is intuitive and meets their needs.

- **Usability Testing:**
 - User interface evaluation
 - Navigation flow testing
 - Feature functionality validation
 - User feedback collection
- **Compatibility Testing:**
 - Cross-browser testing
 - Mobile device compatibility
 - Screen resolution testing
 - Accessibility compliance testing

5.2.6 System Testing:

Testing the entire system as a whole to ensure it meets all requirements.

5.2.7 Black Box Testing:

Focuses on validating the functionality of the system without knowledge of the internal code or structure.

Tests the system against its specified requirements. Example:

- Validate whether a student can successfully register and log in.
- Result report and History Downloaded successfully.

5.2.8 White Box Testing:

Examines the internal structure of the application. Ensures code optimization, coverage, and correctness.

Example:

- Unit testing of individual methods or functions like " Check the URL correct "

5.2 Test Cases

5.2.1 Frontend Testing:

- Test that the URL input field correctly validates URL formats.
- Test that the image upload component accepts valid file types and rejects invalid ones.
- Test that protected routes redirect unauthenticated users to the login page.
- Test that the dashboard correctly displays the user's analysis history.

5.2.2 Backend Testing:

- Test the **/api/auth/register** endpoint with valid and invalid data.
- Test the **/api/predict** endpoint with known phishing and safe URLs to verify the model's accuracy.
- Test that JWT authentication correctly allows and denies access to protected endpoints.
- Test the report generation endpoints to ensure they produce valid PDF and Word files.

5.2.3 Performance Testing:

- Measure the response time of the **/api/predict** endpoint under load.
- Simulate multiple concurrent users to ensure the system remains responsive.
- Test the image processing endpoint with various image sizes to measure processing time.

5.3 Security Testing

- **OWASP Top 10:** The system is tested against common vulnerabilities like SQL Injection, Cross-Site Scripting (XSS), and Broken Authentication.
- **Penetration Testing:** Attempting to bypass authentication and authorization mechanisms.
- **API Security:** Testing for rate limiting, proper error handling that doesn't leak information, and validation of all inputs.

5.4 Test Coverage Report Explanation

- **Test Coverage Report:** A Test Coverage Report is a metric that measures the amount of code that is executed when a test suite runs. It provides insight into which parts of the codebase are tested and which are not, helping teams identify gaps in their testing strategy.
- **Used in This Project:** For the LinkGuard project, the Test Coverage Report is valuable because:
 - It quantifies how thoroughly the codebase has been tested
 - It helps identify untested code that might contain bugs
 - It provides confidence in the reliability of the system
 - It guides future testing efforts to focus on uncovered areas
 - It demonstrates the quality assurance process to evaluators
- **Test Coverage Report for LinkGuard:**

The report should show:

- Overall coverage percentage
- Coverage by module (Authentication, URL Analysis, Image Processing, Report Generation)
- Line coverage, branch coverage, function coverage
- Visual representation (pie chart or bar graph)
- Uncovered critical paths that need additional testing

CHAPTER NO.6

GRAPHICAL USER INTERFACE

6.1 Introduction

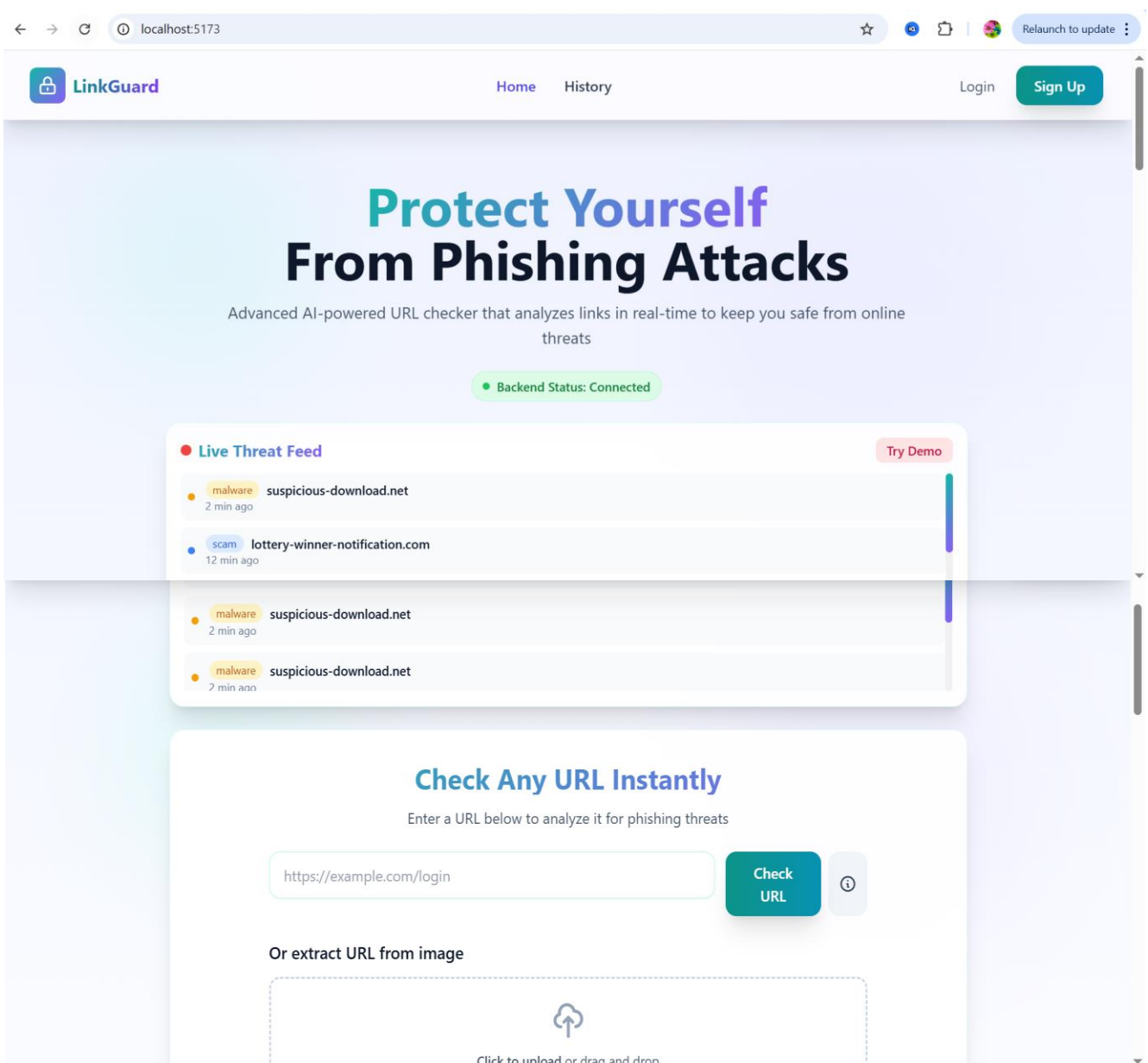
A GUI uses visual elements such as icons, buttons, and windows to allow users to interact with a system. It is prevalent in modern software applications and operating systems.

the main page of the **LinkGuard web-based Application**.

Design Philosophy: The LinkGuard UI is designed around the principles of **Clarity, Trust, and Efficiency**. The interface aims to make sophisticated security analysis accessible to all users, regardless of their technical expertise. It uses a clean, modern design with a professional color palette to inspire confidence and provide clear, immediate feedback.

6.2 Interface Screenshots and Descriptions

6.2.1 Home Page



Link Detection System

PNG, JPG, GIF up to 10MB

Or

Paste from clipboard (Ctrl+V)



How It Works

Our AI analyzes multiple factors including URL structure, domain reputation, and security certificates to provide accurate phishing detection in seconds.

Security Tips



Strong Passwords

Use unique, complex passwords for each account.
Look for secure connections with HTTPS in the address bar



Why Choose LinkGuard?

Discover the features that make us the best choice for your online security



Lightning Fast

Get instant results with our advanced AI-powered analysis

[Click to learn more](#)



Accurate Detection

Industry-leading accuracy in identifying phishing threats

[Click to learn more](#)



Detailed Reports

Comprehensive analysis with confidence scores and explanations

[Click to learn more](#)

Real-Time Statistics



12,543

URLs Checked Today
+12%



1,847

Threats Blocked
+8%



3,291

Active Users
+15%



99.7%

Accuracy Rate
+0.1%

Join LinkGuard Today

Create an account to save your URL checks, view statistics, and get personalized security recommendations.

Login

Sign Up Free



LinkGuard

Protect yourself from phishing attacks with our advanced URL checker. Stay safe online with real-time analysis and detailed reports.



Product

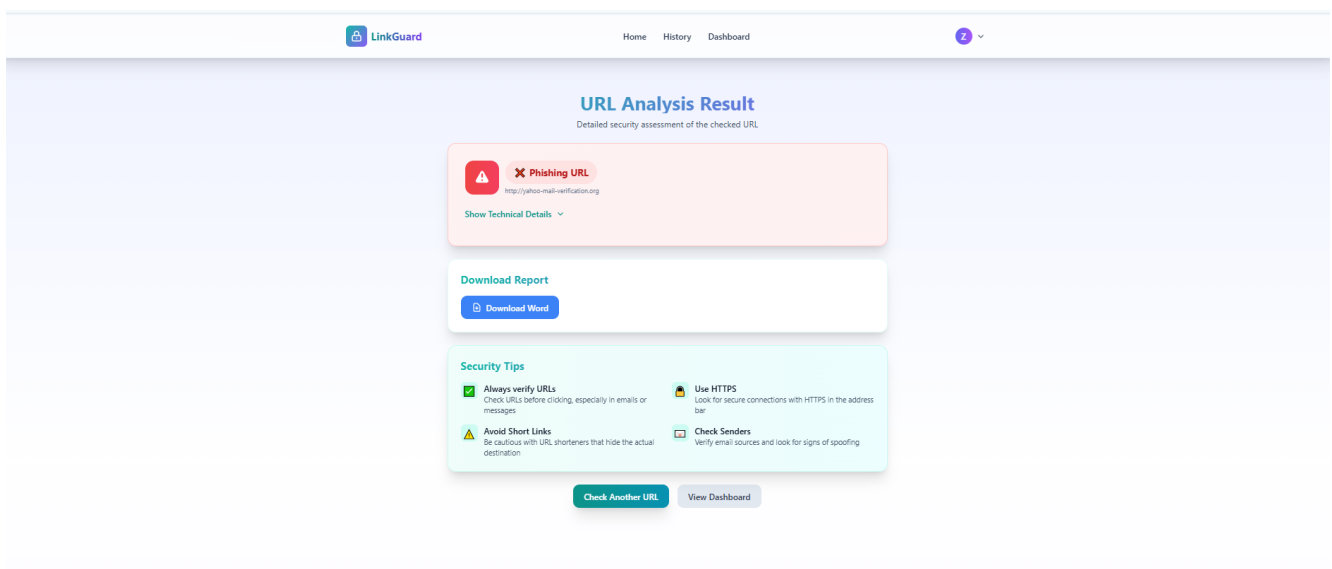
URL Checker
History
Dashboard
API

Resources

Blog
Security Tips
Report Phishing
Contact Us

- **Layout & Structure:** The page features a prominent hero section with the main value proposition. The centerpiece is a large, clean URL input field with a clear "Check URL" button. Below this, there is a section for image upload. The page also includes a simulated "Live Threat Feed" and an interactive "Security Tips" carousel to engage and educate users.
- **Interactive Elements:** The URL input provides real-time validation feedback. The image upload supports drag-and-drop. The threat feed and security tips auto-advance but can also be manually controlled by the user. A "Try Demo" button allows users to see the system in action with a sample phishing URL.
- **Visual Design:** Uses a calming gradient background (teal to blue). The safety verdicts are color-coded (green for safe, red for phishing). Framer Motion is used for subtle animations and smooth transitions, creating a polished feel.

6.2.2 Result Page:



Link Detection System

LinkGuard_Report_20251207_172352 [Compatibility Mode] - Word

Sign in

File Home Insert Draw Design Layout References Mailings Review View Zotero Help Tell me what you want to do Share

Clipboard Font Paragraph Styles Editing Add-ins

LinkGuard Security Report

Report generated for: Zani (zani123@gmail.com)
Date: 2025-12-07 17:23:52

URL Analyzed:
<https://google.com>

Analysis Result: **SAFE**

Confidence Level: 80%

Analysis Details:
ml_safe_prediction

Recommendations:

- ✔ This URL appears to be safe based on our analysis.
- ⚠ Still, always ensure you're on the correct domain before entering sensitive information.
- 🔒 Look for HTTPS in the URL and a padlock icon in your browser's address bar.
- 🚫 Be wary of unexpected pop-ups or requests for additional information.

This report was generated by LinkGuard Phishing Detection System. While we strive for accuracy, no detection system is 100% perfect. Always exercise caution when browsing the Internet.



Home History Dashboard



Multiple URLs Analysis Results

Detailed security assessment of the checked URLs

Summary

3

Safe URLs

0

Phishing URLs

0

Errors

✔ <https://www.microsoft.com> ✔ <https://www.apple.com> ✔ <https://www.khanacademy.org>



Safe URL

<https://www.microsoft.com>

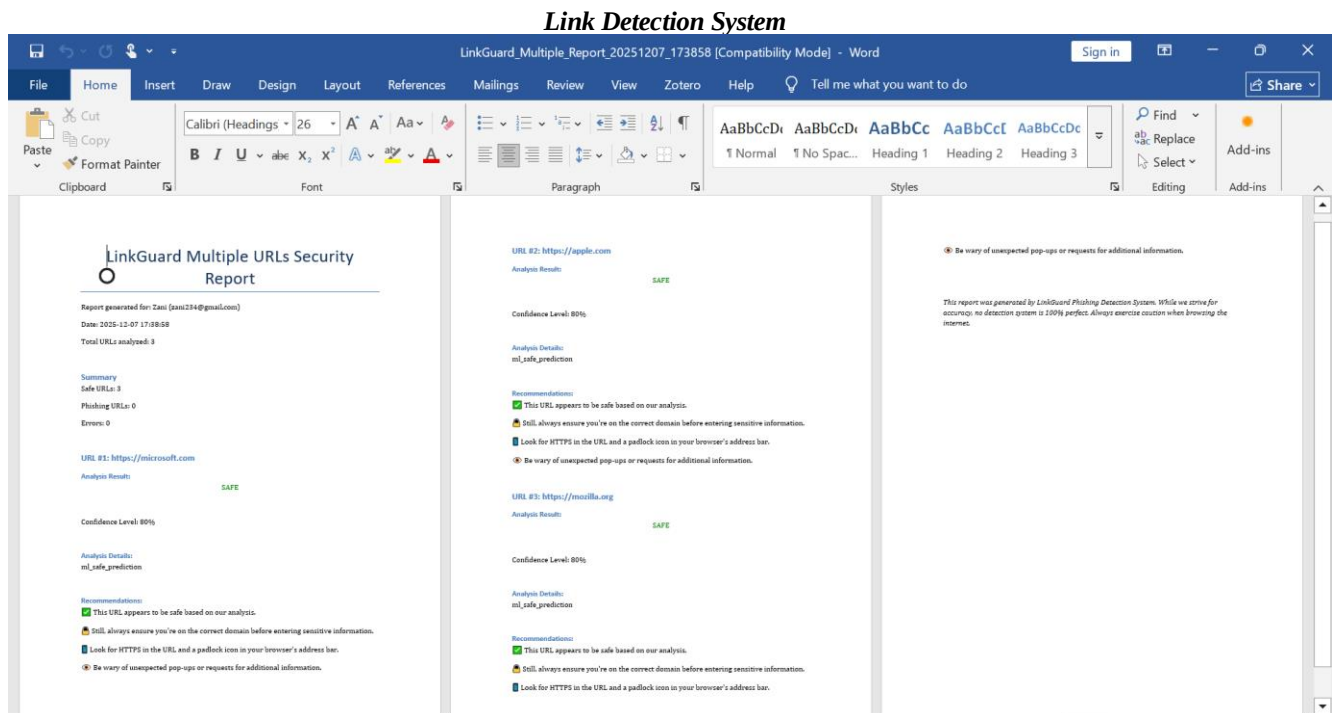
Show Technical Details

Download Report

Download Word

Check More URLs

View Dashboard



- **Layout & Structure:** The page is dominated by a large, clear verdict badge (e.g., "PHISHING DETECTED") with a prominent confidence score. Below this, the analysis is broken down into sections: "ML Model Analysis," "Threat Intelligence Results," and "Security Recommendations."
- **Interactive Elements:** Users can click on "Download Report as PDF" or "Download Report as Word" buttons. Each section of the analysis can be expanded to show more detailed information.
- **Visual Design:** The use of color is critical here. A red verdict immediately signals danger, while green signals safety. Icons are used to represent different threat sources (e.g., a shield for Google Safe Browsing). Charts are used to visualize the confidence score.

6.2.3 Registration Page

This screenshot represents the demo presentation of the User Registration page. To get registered on the Application, users will need to provide their email and personal information. By submitting these details, the user will be able to complete the registration process.

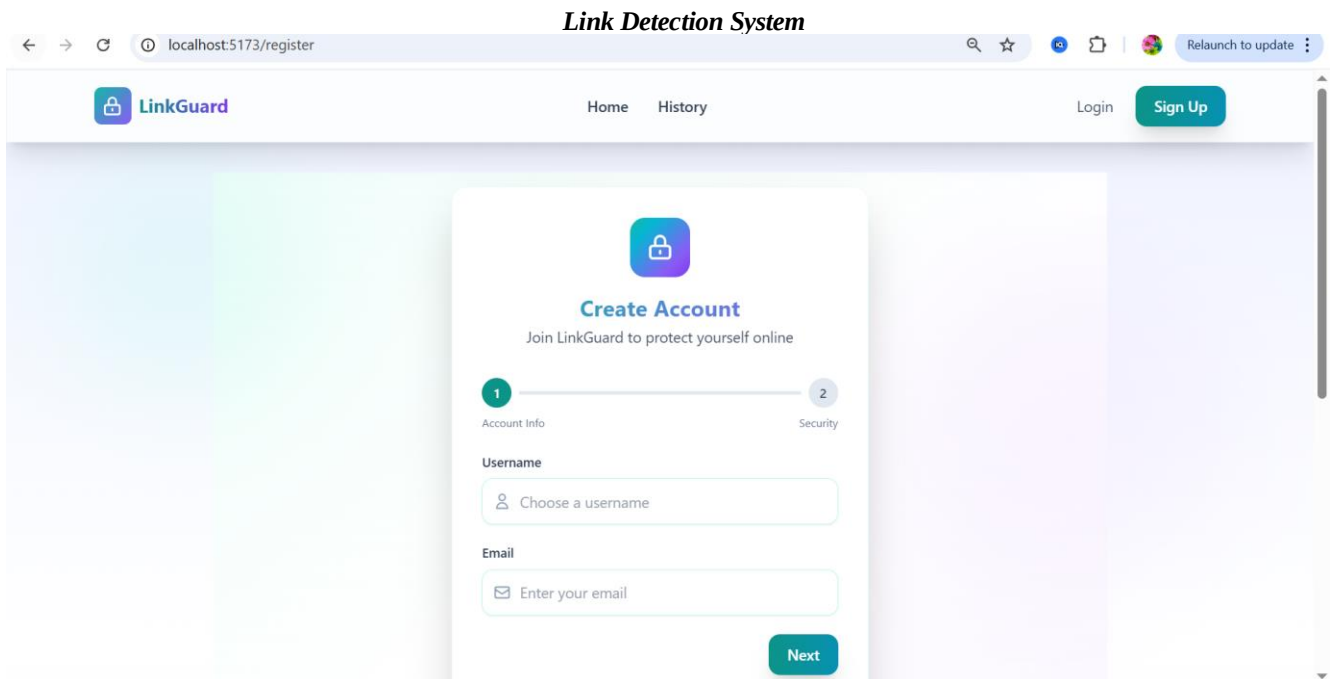


Figure 6.1. Registration Page

6.2.4 Login Page

This screenshot represents the demo presentation of the Login page. To login in Application, user will need to provide their email/username and password. After clicking on login button, user will successfully login in application.

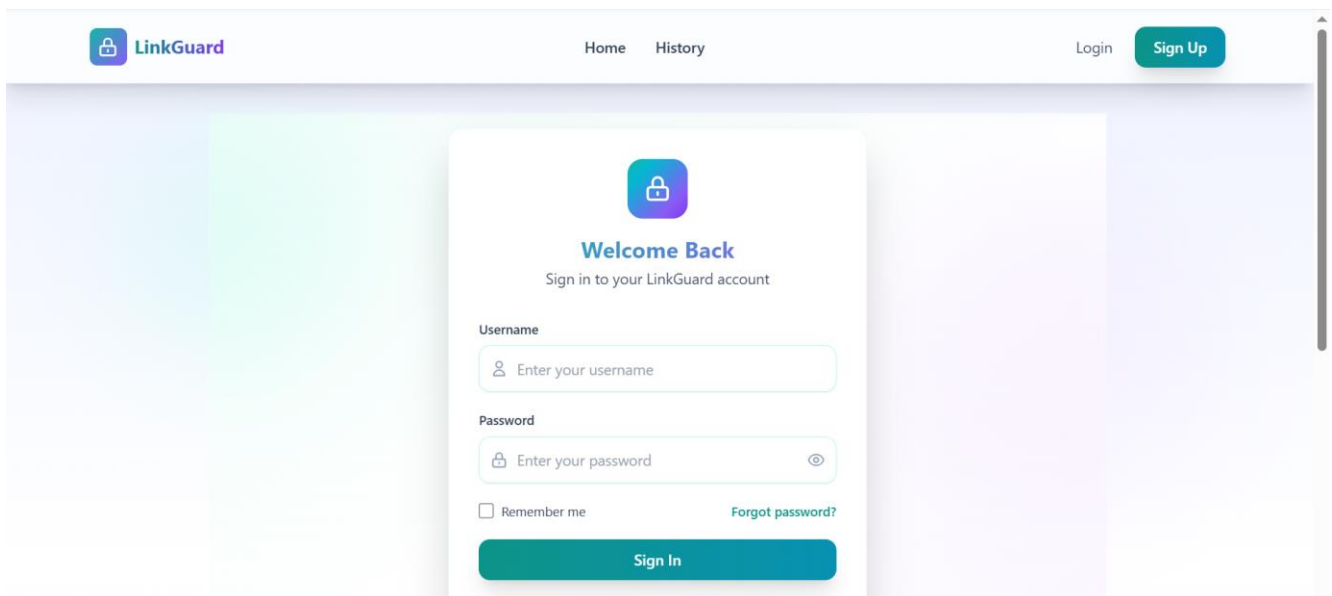


Figure 6.2. Login Page

6.2.5 User Dashboard

Link Detection System



Home History Dashboard



LinkGuard

Overview

Recent Checks

Activity

Settings

Data Management

Zani

zani234@gmail.com

Logout

Welcome, Zani!

Refresh Data

Total Checks
10

Safe URLs
6

Phishing URLs
4

Your Profile

Username
Zani
Member Since
12/7/2025

Email
zani234@gmail.com
Account Status
Active

Check New URL
Analyze a new URL for safety

View History
See all your previous checks



Home History Dashboard



Welcome back,
Gul_Ahmad1009!

Here's your security overview

Quick
Check

Challenge



Overview

Recent

Activity

Achievements

Settings

Total Checks
10
↑ +12%

Safe URLs
9
↑ +8%

Phishing URLs
1
↓ -3%

Your Profile

Username
Gul_Ahmad1009
Member Since
10/18/2025

Email
gullahmad2002@gmail.com
Account Status
Active

Security Score

85%

Your security score is above average

Quick Actions

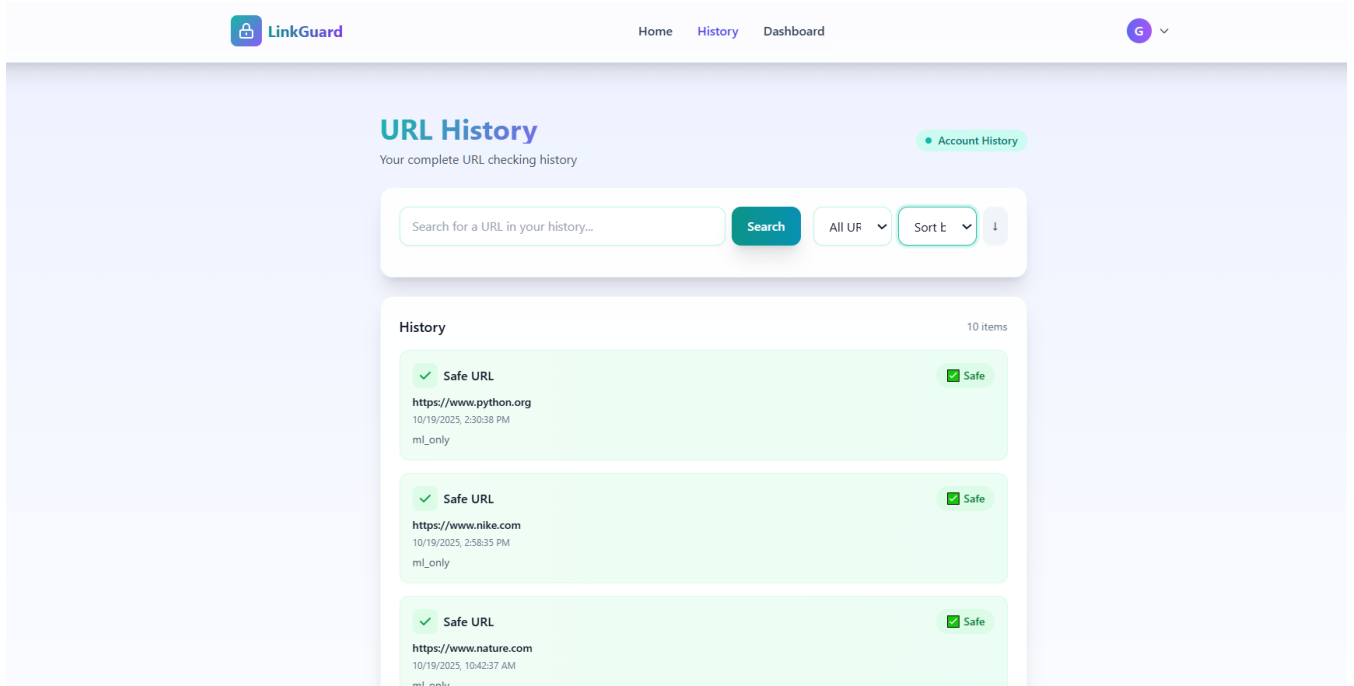
Check New URL
Analyze a new URL for safety

View History
See all your previous checks

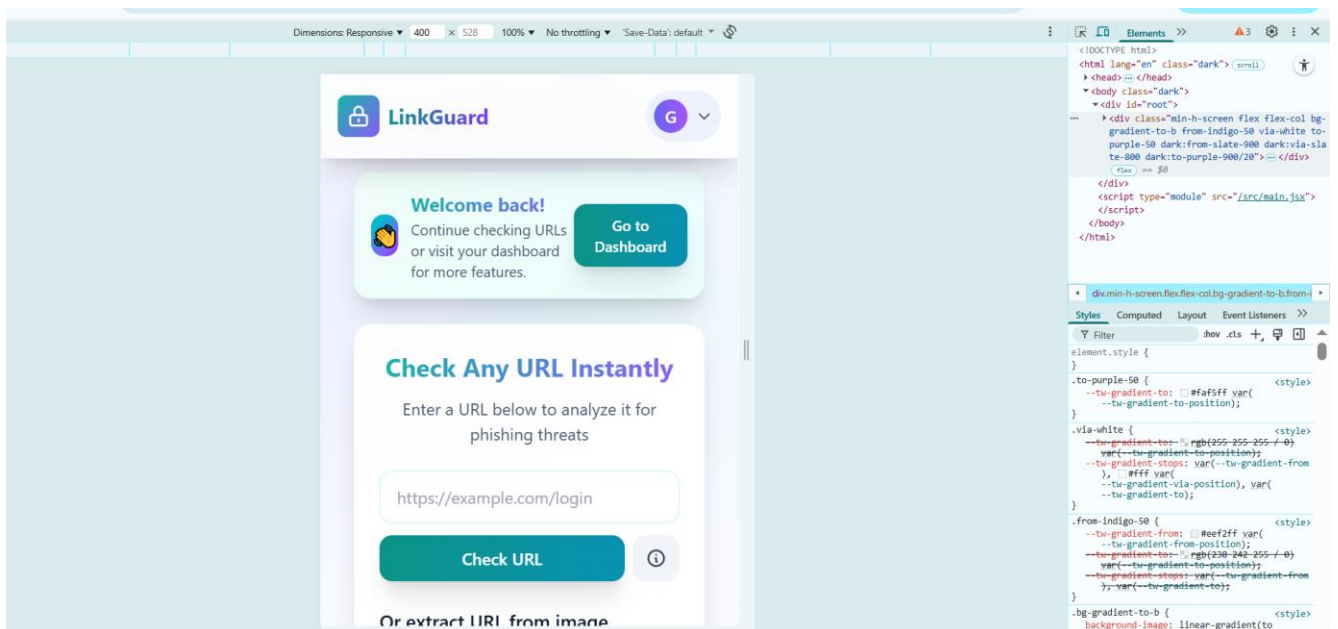
- **Layout & Structure:** The dashboard provides a personalized view. It includes a welcome message, a summary card showing key statistics (e.g., total URLs checked), and a detailed, searchable table of the user's analysis history.
- **Interactive Elements:** The history table has pagination and filtering options. Each row in the table is clickable, taking the user to the detailed results page for that analysis.
- **Visual Design:** A card-based layout is used to organize information neatly. Data visualizations, like a pie chart showing the breakdown of safe vs. phishing URLs, provide at-a-glance insights.

6.2.6 History Page

Link Detection System



6.2.7 Mobile Interface



- **Layout & Structure:** The mobile layout is a single-column, scrollable design. Navigation is handled by a bottom tab bar for easy thumb access. All essential features, like URL checking and viewing history, are fully functional.

Link Detection System

- **Interactive Elements:** All buttons and input fields are optimized for touch with larger tap targets. Swiping gestures can be used for navigation in some areas.
- **Visual Design:** The design is simplified to focus on core functionality. Text is larger, and there is more white space to prevent the interface from feeling cluttered on a small screen.

CHAPTER NO.7

CONCLUSION

7.1 Project Summary

LinkGuard successfully delivers a comprehensive and intelligent solution to the pervasive problem of phishing. By integrating a high-accuracy machine learning model with real-time threat intelligence, the system provides users with a powerful tool to safeguard their digital lives. The project demonstrates a successful fusion of modern web technologies, data science, and user-centric design.

7.1.1 Key Achievements:

- Developed a functional, end-to-end phishing detection system.
- Implemented a hybrid detection approach (ML + Threat Intel) for high accuracy.
- Created an innovative image processing feature for URL extraction.
- Built a professional, responsive, and user-friendly web interface.
- Incorporated a valuable report generation feature for documentation and awareness.

7.2 Future Enhancement

While the current system is robust, there are clear paths for future development:

- **Browser Extension:** Develop a browser extension that automatically checks links as users browse the web, providing real-time warnings without needing to visit the LinkGuard site.
- **Continuous Learning:** Implement a system for the ML model to continuously learn from user feedback and new phishing data, improving its accuracy over time.
- **Corporate Features:** Add features for business teams, such as shared dashboards, bulk URL checking, and phishing simulation training for employees.
- **Email Integration:** Develop a feature where users can forward suspiciously emails to a LinkGuard address, which would then analyze all the links contained within.

CHAPTER NO. 8

REFERENCES

References:

1. **IEEE Standard 830-1998:** "Recommended Practice for Software Requirements Specifications."
2. **Scikit-learn: Machine Learning in Python**, Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.
3. **Flask Web Framework**, <https://flask.palletsprojects.com/>
4. **React - A JavaScript Library for Building User Interfaces**, <https://reactjs.org/>
5. **Google Safe Browsing API Documentation**, <https://developers.google.com/safe-browsing>
6. **VirusTotal API Documentation**, <https://developers.virustotal.com/>
7. **OWASP Top Ten Web Application Security Risks**, <https://owasp.org/www-project-top-ten/>
8. **Tesseract OCR Documentation**, <https://tesseract-ocr.github.io/>

Appendix

A. Tools and Technologies Used

- Frontend: React.js, Vite, Tailwind CSS, Framer Motion, Axios, React Router.
- Backend: Python, Flask, SQLAlchemy, Scikit-learn, Pillow, OpenCV, Tesseract, ReportLab, python-docx.
- Database: SQLite (for development), with easy migration to PostgreSQL.
- External APIs: Google Safe Browsing, VirusTotal, URLhaus, WHOIS.